Logical Clocks (Lamport)

ordering by cause, not by the time of day

Watch the 4-min video

In one line: a Lamport clock is a counter on each node, incremented on every event and carried on every message, so if event a could have caused b, then a's timestamp is smaller than b's; ties broken by node ID give a total order every replica agrees on, but timestamps alone can't tell you whether two events were concurrent.

1 Order without real time

If wall clocks can't order events across machines (lesson 5), what can? In 1978 Leslie Lamport showed that you don't need real time at all. What matters is **causality**: which events could have influenced which.

2 Happens-before

Event *a* **happens before** event *b* (written a → b) if: both are on the same process and a comes first; or a is the sending of a message and b is its receipt; or there's a chain of such steps from a to b. If neither a → b nor b → a, the events are **concurrent**: neither could have known about the other.

3 The Lamport clock

Each process keeps one integer counter. It increments the counter for each local event, attaches the counter to each message it sends, and on receiving a message sets its counter to `max(local, received) + 1`. Result: **if a → b, then L(a) < L(b)**. The converse is not true: a smaller timestamp doesn't prove causality.

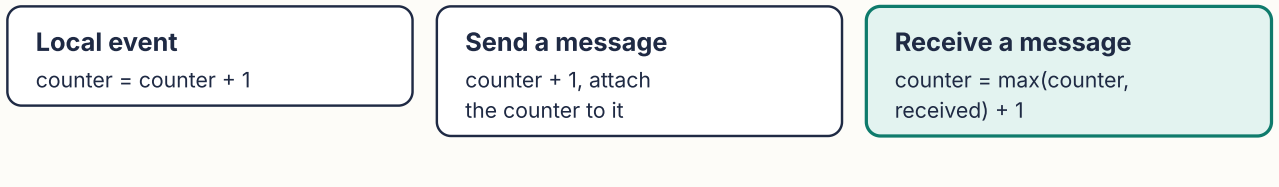


Figure 1. Three rules are the whole algorithm.

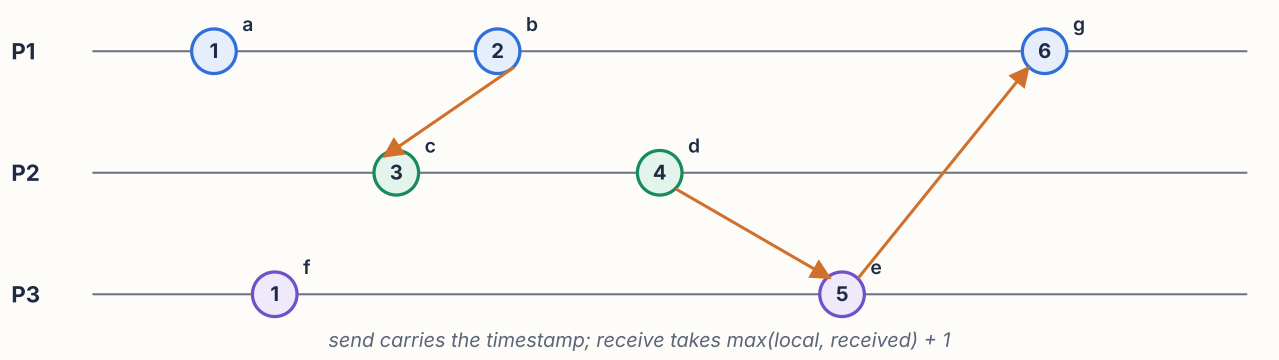
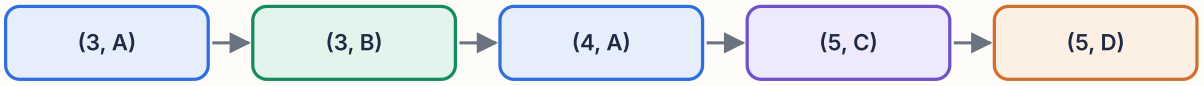


Figure 2. Lamport timestamps on three processes. a → b → c → d → e → g, so their numbers increase. f (timestamp 1) is concurrent with all of P1's and P2's early events.

4 From partial order to total order

Two events can share a timestamp. Break ties with the process ID, ordering by the pair **(timestamp, node ID)**. Now every pair of events has a definite order, and every node that sorts the same events gets the **same sequence**. That order respects causality, though for concurrent events it is arbitrary.



sort by (timestamp, node ID): every replica gets the same total order

Figure 3. Pairs (timestamp, node) give one sequence all replicas agree on.

WORKED EXAMPLE: ORDERING REPLICATED LOG ENTRIES

Three replicas accept operations and exchange them. Each entry carries a Lamport timestamp and the ID of the replica that created it. Replica A creates `set x=1` at (3, A); replica B, which had already received A's earlier messages, creates `set x=2` at (4, B). Every replica applies entries sorted by (timestamp, ID), so all end with x=2, and an operation that depends on another is always applied after it. Lamport's paper used exactly this idea to build a mutual-exclusion algorithm with a shared request queue.

5 Limits

- Lamport timestamps can't detect **concurrency**: L(a) < L(b) might mean a caused b, or that they were unrelated. For that you need vector clocks (lesson 7).
- They have no link to real time, so 'what happened at 3 p.m.?' can't be answered.
- The total order is fixed only after you've heard from everyone, so it alone doesn't give fault-tolerant agreement; that needs consensus (lesson 9).

COMMON MISTAKES

- Concluding 'a caused b' from L(a) < L(b).
- Forgetting to update the counter on receive, which breaks the guarantee.
- Comparing timestamps without a tie-breaker, so replicas disagree on equal values.

6 Key terms

Happens-before (→): a could have influenced b

Lamport clock: per-node counter carried on messages

Tie-breaker: node ID used when timestamps are equal

Concurrent events: neither happened before the other

Total order: every pair of events is ordered the same way everywhere

7 Interview questions

Q1. What guarantee does a Lamport clock give? BASIC

If event a happens before b, then a's timestamp is smaller than b's. It does not guarantee the reverse.

Q2. How do you update a Lamport clock on receive? BASIC

Set it to the maximum of the local counter and the timestamp in the message, then add one.

Q3. Three replicas must apply the same operations in the same order. How can Lamport clocks help? SCENARIO

Stamp each operation with (Lamport time, replica ID) and apply in sorted order; causally related operations stay in order and all replicas agree on the rest.

Q4. You need to know whether two edits conflicted. Are Lamport timestamps enough? SCENARIO

No: they can't distinguish concurrent edits from causally ordered ones. Use version vectors or vector clocks to detect concurrency.

Q5. Why not just use synchronised physical clocks? TRICKY

Skew means a causally later event can get an earlier timestamp. Lamport clocks cost one integer and never violate causality.

Q6. What's the 'clock condition'? TRICKY

For any events a → b, C(a) < C(b). Lamport clocks satisfy it; vector clocks satisfy the stronger version where the converse also holds.

Next: SD-H-007 · Vector Clocks and Hybrid Logical Clocks backendroadmap.com